

Análise de Sistemas — Resumo Consolidado para Prova

Engenharia de Software — 2º período

1. Visão geral da disciplina

A disciplina usa o paradigma de **Orientação a Objetos (OO)** e a **UML** para ensinar Análise de Sistemas, sempre com analogias de jogos (RPG, e-sport, MMO). O conteúdo se organiza em 5 blocos:

- Fundamentos de OO e Abstração
- Modelagem de Classes de Análise (UML 2.0)
- Modelagem de Casos de Uso (MCU)
- Mecanismos Gerais da UML
- Processo de Desenvolvimento de Software (PDS)

UML: linguagem visual de modelagem, **independente de linguagem de programação e de processo de desenvolvimento**. Não é uma linguagem de programação.

- Diagramas estruturais** = visão estática (classes, componentes, implantação).
- Diagramas comportamentais** = visão dinâmica (casos de uso, sequência, atividades, estados).

2. Fundamentos de Orientação a Objetos

Conceito	Definição
Classe	Molde genérico e estático. Agrupa atributos e operações comuns de uma entidade. Nunca "faz" nada sozinha (não é correto dizer que a classe "joga" ou "recebe dano").
Objeto	Instância concreta e ativa em execução, com estado real (analogia: "arquivo de Save Game").
Abstração	Selecionar apenas os dados relevantes ao contexto da aplicação, descartando o resto. Um modelo não deve representar todos os detalhes do sistema real — isso contraria a abstração.
Encapsulamento	Ocultar implementação interna (caixa-preta) e expor só a interface. Quem usa o objeto sabe <i>o quê</i> pedir, não <i>como</i> é processado por dentro.
Polimorfismo	Mesma mensagem, respostas diferentes conforme o objeto que a recebe (ex.: <code>interagir()</code> em Porta, NPC ou Baú produz efeitos distintos).
Herança (Generalização)	Hierarquia do mais geral para o mais especializado; subclasses herdam atributos/operações da superclasse e adicionam os próprios. Promove reuso lógico.
Mensagem / Colaboração	Objetos colaboram trocando estímulos. Sequência: gatilho → mensagem/estímulo → processamento pelo receptor → mudança de estado .

Classe ≠ Objeto — não são sinônimos. Classe é o molde; objeto é o exemplar vivo com dados.

Exemplo de abstração contextual: o mesmo *carro* é modelado diferente conforme a aplicação —

- Simulador de corrida: motor, arrasto, desgaste dos pneus, suspensão, velocidade.
- Construtor de cidades: só posição no mapa, direção e velocidade geral (dados de motor/suspensão seriam desnecessários aqui).

Exemplo de cadeia de mensagens (batalha): (1) jogador aperta o botão de ataque → (2) `Jogador` envia `calcularDano()` ao `Inimigo` → (3) `Inimigo` processa o estímulo → (4) `Inimigo` perde vida (muda de estado).

3. Modelagem de Classes de Análise (UML 2.0)

3.1 Regra de Ouro da Análise

Na fase de **Análise**, o foco é o domínio do negócio (**o quê** o sistema faz) — não interessa banco de dados, tabelas relacionais ou código de implementação (**o como**, que pertence à fase de Projeto/Implementação). Grady Booch alerta contra o engenheiro amador que busca "mágicas"/panaceias em vez de entender o domínio.

Regra de Ouro da Modelagem (existência de classe): se um elemento não possui atributos, não realiza ações e não tem responsabilidades claras no domínio, ele não deve existir como classe.

3.2 Anatomia da classe (3 compartimentos)

- 1. **Nome** — o conceito do domínio (ex.: `Personagem`).
- 2. **Atributos** — dados que caracterizam a classe (ex.: `pontosDeVida` , `nível`).
- 3. **Operações** — comportamentos que a classe sabe realizar (ex.: `atacar()` , `curar()`).

No nível de análise, não importa a fórmula/código interno de uma operação — só o que ela representa para o negócio.

3.3 Diagrama de Classes x Diagrama de Objetos

Critério	Diagrama de Classes	Diagrama de Objetos
Natureza	Estático — "manual/livro de regras"	Dinâmico — "fotografia/save game" de um instante
O que mostra	O que pode existir (estrutura)	O que existe de fato (instâncias com valores reais)
Notação do nome	Texto simples: <code>NomeClasse</code>	Sublinhado: <code>nomeObjeto : NomeClasse</code>

3.4 Associação e Multiplicidade

A multiplicidade define restrições numéricas rígidas de participação — funciona como a "lei física" do sistema, evitando inconsistências.

Notação	Significado	Tipo de participação
<code>1..1</code>	Exatamente um	Obrigatória e única
<code>0..1</code>	Zero ou um	Opcional e única
<code>1..*</code>	Um ou muitos	Obrigatória e múltipla
<code>0..*</code>	Zero ou muitos	Opcional e múltipla

Regra de leitura bidirecional: a multiplicidade deve ser lida nos dois sentidos da associação — da origem para o destino e do destino de volta para a origem. Cada sentido pode ter uma multiplicidade diferente.

Exemplo correto: 1 `Jogador` cria de 1 a muitos (`1..*`) `Personagens` . Na volta, 1 `Personagem` pertence a exatamente 1 (`1..1`) `Jogador` .

Erro de inversão: se a volta fosse `Personagem` `[1..*]` `Jogador` , um mesmo personagem poderia ser controlado por vários jogadores ao mesmo tempo — inconsistência grave.

Notação	Domínio RPG/Jogos	Domínio corporativo
<code>1..1</code>	Personagem ↔ Ficha Base/Raça	Pedido ↔ Nota Fiscal
<code>0..1</code>	Personagem ↔ Montaria/Arma equipada	Cliente ↔ Cartão de Fidelidade
<code>1..*</code>	Party (grupo) ↔ Membro	Conta Bancária ↔ Movimentações
<code>0..*</code>	Personagem ↔ Itens no Inventário	Produto ↔ Avaliações de Usuários

3.5 Agregação x Composição

Critério	Agregação (Compartilhada)	Composição (Não-Compartilhada)
Símbolo	Losango vazado na ponta do "Todo"	Losango preenchido na ponta do "Todo"
Relação	"Tem-um"	"É-parte-de"
Ciclo de vida	Independente — destruir o Todo não destrói a Parte	Dependente — destruir o Todo destrói as Partes
Compartilhamento	A Parte pode pertencer a vários Todos	A Parte pertence a um único Todo
Exemplo	Guilda-Personagem: se a guilda acaba, o personagem sobrevive	Personagem-Inventário: se o personagem é deletado, o inventário some junto

3.6 Herança e os 3 erros fatais de associação/herança

Herança (Especialização): relação "é-um", transitiva e **anti-simétrica** — precisa respeitar o **Princípio de Substituição de Liskov** (a subclasse deve poder substituir a superclasse sem quebrar a consistência do sistema).

3 erros clássicos que a prova costuma cobrar: 1. **Confundir herança com associação** — ex.: `Gerente` não "é-um" `Departamento` ; ele *chefia/associa-se* a um departamento. Correto = associação, não generalização. 2. **Violar Liskov** — ex.: `ControleDeXbox` não pode ser subclasse de `Teclado` , mesmo ambos sendo periféricos (não passam no teste de substituição). 3. **Vazar tecnologia/infraestrutura para a Análise** — ex.: atributo `VARCHAR(100)` ou método `conectarMySQL()` dentro de uma classe de análise.

3 erros fatais especificamente em multiplicidade/associação: 1. Inverter as pontas do asterisco (colocar `1..*` na origem errada) → causa estouro de capacidade no sistema. 2. Confundir `0..1` com `1..1` → força obrigatoriedade onde deveria ser opcional, travando a navegação do usuário. 3. Usar herança em vez de associação em relações de posse/uso (ex.: `Guerreiro` herdar de `Espada` em vez de "ter/usar" uma espada).

3.7 Técnicas de identificação de classes

Análise Textual de Abbott (mapeamento gramatical de requisitos em texto natural):

Elemento gramatical	Vira em OO
Substantivos	Classes/objetos candidatos
Adjetivos	Atributos
Verbos de ação	Operações/métodos
Verbos ter/pertencer	Agregação ou Composição
Verbos ser	Herança (Generalização)

Limitação de Abbott: falha diante de textos ambíguos e sinônimos.

Estereótipos BCE (Boundary-Control-Entity) — organiza classes de análise em 3 papéis:

- «boundary» **(Fronteira)** — interface com atores externos (telas, formulários, APIs).
- «control» **(Controle)** — orquestra o fluxo de execução e a lógica de coordenação.
- «entity» **(Entidade)** — dados persistentes e regras centrais do negócio.
- Vantagem:** resiliência — mudar a plataforma (PC → Mobile) afeta só a classe de Fronteira, sem tocar em Controle/Entidade.

Cartão CRC (Class, Responsibility, Collaboration): registra o **quê** a classe faz (responsabilidades, não como) e **com quem** ela colabora para cumprir isso.

Visão de Classes Participantes (VCP): diagrama que mapeia só as classes necessárias para **um único caso de uso** ("mapa tático de uma missão"). Evita o antipattern do **"Diagrama de Classes Deus"** (modelo monolítico com centenas de classes soltas, sem divisão lógica).

Outros erros a evitar na modelagem de classes: instanciar conceitos abstratos genéricos sem subclasses concretas; criar ciclos em hierarquias de herança.

4. Modelagem de Casos de Uso (MCU)

Objetivo do MCU: representar os requisitos funcionais **externamente observáveis** sob a ótica do ator, validando o escopo com o cliente antes de programar — evita o **"Abismo"** (quando o entregue ≠ o pedido).

4.1 Componentes principais

- Ator:** papel externo (humano, outro sistema ou hardware) que troca mensagens com o sistema. **Nunca é uma pessoa específica** — é o papel que ela exerce (ex.: "João da Silva" está errado; o certo é modelar os papéis "Moderador" e "Jogador" que ele desempenha em momentos diferentes).
- Fronteira do sistema:** limite gráfico entre o que é interno (software) e externo (ambiente). Um elemento só é ator se **cruzar essa fronteira**. Um NPC que só interage com rotinas internas da engine, sem trocar mensagens externas, **não é ator**.
- Caso de uso:** sequência de interações que gera valor mensurável ao ator; nomeado com **verbo no infinitivo + substantivo**.
- Primário:** objetivo principal do ator, traz valor direto.
- Secundário:** processo administrativo/suporte/manutenção, sem valor direto ao usuário final.

4.2 Relacionamentos entre casos de uso

Relacionamento	Natureza	Direção da seta	Regra
<<include>>	Obrigatória	Aponta para o caso incluído	O caso base sempre aciona o incluído — nunca é condicional
<<extend>>	Condicional/opcional	Aponta para o caso base	Só é acionado sob condição específica (ponto de extensão)
Generalização	Herança	—	Aplicável tanto a atores quanto a casos de uso

Exemplo <<include>> : `Iniciar Partida Ranqueada` sempre inclui `Verificar Conexão` (obrigatório). **Exemplo <<extend>> :** `Explorar Masmorra` pode ser estendido por `Encontrar Chefe Secreto`, só se o jogador tiver um item raro (condicional).

Notação gráfica: atores ficam **fora** do retângulo da fronteira; casos de uso são **elipses**; a fronteira é um retângulo nomeado com o sistema.

4.3 Regras de negócio, desempenho e priorização

- Regra de Negócio [RN]:** política/restrição de negócio (ex.: "um jogador não pode pertencer a mais de uma guilda").
- Requisito de Desempenho [RD]:** métrica técnica/SLA (ex.: "emparelhamento em até 5 segundos").

Matriz de Priorização (Risco Técnico × Prioridade) — usada no planejamento de sprints:

Alta Prioridade		Baixa Prioridade
Alto Risco	Atacar Primeiro	Próximo Alvo
Baixo Risco	Desenvolver Depois	Deixar para o Final

Regra de Sobrevivência: itens de alto risco técnico devem ser atacados nas **primeiras sprints**, nunca postergados.

4.4 Regra dos 100 Anos (Descrição Essencial)

A especificação de um caso de uso deve focar na **intenção de negócio pura** do ator, sem se acoplar a tecnologia, telas, botões físicos ou código — a descrição deveria continuar válida "daqui a 100 anos", independente da tecnologia da época.

Especificação textual de caso de uso deve conter: Nome do Caso de Uso, Ator Primário, Objetivo, e passos numerados no formato **Sujeito + Predicado** (com marcações de [RN]/[RD] quando aplicável).

4.5 Antipatterns de Caso de Uso (erros clássicos de prova)

Erro	Exemplo errado	Correção
Nome com implementação técnica	"Processamento Interno do Algoritmo de Busca via Netcode SQL"	Nomear pela ótica do ator primário (ex.: "Encontrar Partida")
Violação da Regra dos 100 Anos	"Jogador aperta botão X no DualSense", "menu em resolução 4K", <code>apply_heal(player_id, 50)</code>	Descrever de forma essencial: "jogador seleciona a poção", sem hardware/código
<code><<include>></code> usado como condicional	"às vezes o bônus é incluído na compra"	Isso é <code><<extend>></code> , não <code><<include>></code> (que é sempre obrigatório)

5. Mecanismos Gerais da UML ("Toolkit do Analista")

Criados porque os elementos básicos da UML (classes, atores, componentes) não bastam para documentar infraestrutura, hierarquias grandes e regras matemáticas rígidas — sem precisar inventar diagramas novos.

Mecanismo	Função	Notação
Estereótipos	Estendem/alteram o significado padrão de um elemento	Gráfica (ícone) ou rótulo textual <code>«nome»</code>
Etiquetas (Tags)	Metadados customizados	<code>{ chave = valor }</code> (ex.: <code>{autor = "...", data = "..."} </code>)
Notas Explicativas	Comentários em texto livre sobre decisões/regras	Retângulo com dobra no canto superior direito, ligado por linha pontilhada
Pacotes	Agrupamento lógico de artefatos/subsistemas	Ícone de pasta com aba superior
Restrições / OCL	Regras formais e não ambíguas	Estrutura tripartite (ver 5.2)

5.1 Pacotes: visibilidade e dependência

Símbolo	Visibilidade	Acesso
<code>+</code>	Pública	Acessível livremente de fora do pacote
<code>#</code>	Protegida	Acessível só dentro da hierarquia de pacotes
<code>-</code>	Privativa	Bloqueada para acesso externo, uso interno exclusivo

Regra de Dependência entre Pacotes: P1 depende de P2 quando uma alteração em P2 pode impactar P1. Dependências excessivas, desordenadas ou circulares comprometem a manutenibilidade a longo prazo.

5.2 OCL (Object Constraint Language)

Estrutura obrigatória em 3 partes, nesta ordem: 1. **Contexto** — o elemento do modelo ao qual a regra se aplica. 2. **Propriedade** — o atributo/associação sendo restringido. 3. **Operação** — a condição lógica/matemática (pré-condição, pós-condição ou invariante), usando operadores como `>=`, `<=`, `and`, `or`, `implies`, `not`.

Regra de negócio	Contexto	Propriedade	Operação
Lançar magia exige mana suficiente	Ação de lançar magia	Pontos de mana do jogador	<code>mana_atual >= custo_do_feitiço</code>
Conta bancária exige maioridade	Criação de Conta	Idade do Cliente	<code>idade >= 18</code>

Quando usar Nota livre em vez de OCL: quando **não** há necessidade de rigor matemático — regras qualitativas, decisões de arquitetura, efeitos sonoros/visuais. OCL é reservado para regras financeiras, numéricas e validações estritas.

5.3 Antipatterns dos Mecanismos Gerais

Erro	Correção recomendada
Regras matemáticas rígidas (juros, multas) descritas só em texto livre	Formalizar com OCL
Diagrama de classes "achatado", com 200+ classes soltas sem divisão	Organizar em Pacotes
Elementos sensíveis (senhas, chaves, persistência) com visibilidade pública (+)	Tornar privativos (-)
Cores/ícones personalizados não documentados para diferenciar classes	Usar Estereótipos padronizados da UML

6. Processo de Desenvolvimento de Software (PDS)

PDS: conjunto estruturado de atividades, métodos e papéis para definir, desenvolver, testar e manter um software — evita o **"Caos sem Regras"** (cada programador codificando sem planejamento).

6.1 Papéis (analogia "Party de RPG")

Papel na Party	Papel Técnico
Líder que distribui as missões	Gerente de Projeto
Dano/DPS que cria o código bruto	Programador
Curandeiro que tira os bugs e salva a equipe	Avaliador de Qualidade (QA)
NPC VIP de escolta (se não for protegido, a missão falha)	Cliente/Usuário — participação indispensável/inegociável
Define estrutura técnica e escolhe arquitetura/engine	Arquiteto/Projetista

Também citados como participantes: **Analista** e **Projetista**, com funções específicas de análise e projeto dentro da equipe.

6.2 Atividades típicas do PDS

Levantamento → Análise → Projeto → Implementação → Testes → Implantação. (Exemplo dado em aula: Levantamento = "ouvir jogadores".)

6.3 Cascata × Iterativo/Incremental

Dimensão	Modelo em Cascata	Modelo Iterativo e Incremental
Progressão	Sequencial e rígida, etapa por etapa	Ciclos repetitivos de mini-cascatas
Requisitos	Totalmente conhecidos e congelados no início	Flexíveis, refinados a cada ciclo
Entregas	Só no final	Incrementos funcionais a cada ciclo
Risco de erro	Altíssimo — erro inicial propaga e custa caro para corrigir	Baixo — erros detectados e corrigidos rapidamente
Analogia	"Modo Hardcore": jogo linear sem save points	"Acesso Antecipado" (Early Access)

Propagação de erro na Cascata: um erro introduzido nos Requisitos (Level 1) e só descoberto nos Testes (Level 4/5) custa exponencialmente mais caro do que se fosse detectado no início — contamina análise, projeto e código ao longo do caminho.

Iterativo x Incremental (definição precisa):

- Iterativo** = repetir ciclos completos (mini-cascatas) para **refinar** com base em feedback.
- Incremental** = **adicionar** novas funcionalidades (incrementos) a cada ciclo.

6.4 Gestão de riscos e prototipagem

- "Target First" / "Enfrentar o Chefão"**: atacar o maior risco técnico logo no primeiro ciclo, nunca deixar para o final.

Citação de **Tom Gilb**: "Se você não atacar os riscos do projeto ativamente, então estes irão ativamente atacar você."

- Prototipagem (Cenário Greybox / Level Design)**: cria versões preliminares para esclarecer incertezas de requisitos (ex.: navegação de interface) **antes** da implementação definitiva. **O protótipo aprovado não substitui a modelagem UML** nem deve virar diretamente o produto final de produção.
- Ferramentas CASE** (Computer Aided Software Engineering): automatizam e apoiam Modelagem, Gerenciamento, Engenharia, Controle, Testes e Desempenho. (As funções detalhadas de cada categoria específica não constam nos materiais disponíveis — checar slide de aula.)
- "Névoa de Guerra" (Fog of War) / Minimapa**: metáfora para a evolução dos diagramas UML nos ciclos de um projeto iterativo/incremental — no Ciclo 1 o modelo é incompleto/nebuloso e se revela/detalha nos ciclos seguintes.

Nota de lacuna: os percentuais exatos do **Chaos Report 1994** (crise do software) e a definição textual formal de PDS com os "2 pilares/dimensões" citados em exercício **não constam em nenhum dos materiais analisados** — são dados apresentados só em aula/slide. Não estude um número que não foi confirmado.

7. Pegadinhas mais prováveis na prova

Afirmação/Situação	Resposta correta
"Um modelo deve representar todos os detalhes do sistema real"	Falso — abstração exige selecionar só o relevante
"A UML é uma linguagem de programação"	Falso — é linguagem visual de modelagem, independente de código
"Classe e objeto são sinônimos"	Falso — classe é molde, objeto é instância viva
Ator nomeado como "João da Silva"	Erro — atores são papéis (Moderador, Jogador), não pessoas
NPC que só interage com rotinas internas da engine	Não é ator — não cruza a fronteira do sistema
Gerente herdando de Departamento	Erro de herança — deveria ser associação ("chefia")
ControleDeXbox subclasse de Teclado	Viola Liskov — não passa no teste de substituição
Atributo VARCHAR(100) ou método conectarMySQL() em classe de análise	Erro — vazamento de infraestrutura para a Análise
<<include>> descrito como condicional ("às vezes")	Erro — condicional é <<extend>>, não <<include>>
Nome de caso de uso citando tecnologia interna ("via Netcode SQL")	Erro — nomear pela ótica do ator, não da implementação
Descrição de fluxo citando botão físico/resolução/código	Viola Regra dos 100 Anos — deve ser abstrata/essencial
Protótipo aprovado vira produto final direto, descartando UML	Erro — protótipo não substitui modelagem UML
Maior risco técnico deixado para o final do projeto	Erro de gestão de risco — deveria ser "Target First"
Elemento sensível (senha, chave) com visibilidade +	Erro de encapsulamento de pacote — deveria ser -
Inverter as pontas de multiplicidade (1..* na origem errada)	Erro fatal de associação — gera inconsistência/estouro
Forçar 1..1 onde deveria ser 0..1	Erro — trava participação que deveria ser opcional

8. Cheat sheet final (revisão de 5 minutos)

- OO:** Classe (molde) × Objeto (instância) | Abstração (só o relevante) | Encapsulamento (interface × implementação) | Polimorfismo (mesma msg, resposta diferente) | Herança (geral → especializado, respeita Liskov).
- Classes de Análise:** Regra de Ouro = negócio, não banco/código | 3 compartimentos (nome/atributos/operações) | Multiplicidade 1..1 / 0..1 / 1..* / 0..*, sempre ler **ida e volta** | Agregação (losango vazado, independente) × Composição (losango cheio, dependente) | Abbott (substantivo→classe, adjetivo→atributo, verbo ação→método, verbo ter→agregação/composição, verbo ser→herança) | BCE (boundary / control / entity) | CRC (o quê + com quem) | VCP evita "Diagrama Deus".
- Casos de Uso:** MCU evita o Abismo | Ator = papel que cruza a fronteira | <<include>> obrigatório → aponta para o incluído | <<extend>> condicional → aponta para o base | RN (regra de negócio) × RD (desempenho) | Matriz de Priorização (risco × prioridade) | Regra de Sobrevivência = risco alto cedo | Regra dos 100 Anos = descrição sempre essencial.
- Mecanismos Gerais:** Estereótipos, Etiquetas, Notas, Pacotes, OCL | Visibilidade + / # / - | Dependência entre pacotes = mudança em P2 afeta P1 | OCL = Contexto + Propriedade + Operação.
- PDS:** Party de RPG (Líder=Gerente, DPS=Programador, Curandeiro=QA, NPC VIP=Cliente, Arquiteto=arquitetura) | Cascata = sequencial, sem save points, erro caro se tardio | Iterativo (refina) × Incremental (adiciona) | Target First = maior risco no Ciclo 1 (Tom Gilb) | Protótipo Greybox não substitui UML | CASE = Modelagem/Gerenciamento/Engenharia/Controle/Testes/Desempenho | Névoa de Guerra = UML se revela nos ciclos.